GameTracker: Aplicativo Flutter de Acompanhamento de Jogos

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps - Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	GameTracker: Aplicativo Flutter de Acompanhamento de Jogos
Gerente de Projeto	Enzo Camelo (Desenvolvedor Líder)
Gerente/Responsável de GCS-DevOps	Enzo Camelo (Desenvolvedor Líder)
Fábrica de Software / Squad	Equipe de 2 pessoas: Enzo-Camelo (Desenvolvedor Líder) e camelo-dev (par de revisão)
Repositório principal	https://github.com/Enzo-Camelo/GameTracker

Histórico de Revisões

Data	Versão	Descrição	Autor
27/10/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD), adaptado ao projeto GameTracker.	Enzo Camelo

Sumário

Sumário **2**
1. Introdução **3**
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 3
1.5 Visão Geral 4
2. Cultura DevOps e Papéis na Gerência de Configuração **4**
3. Ferramentas, Ambientes e Infraestrutura **4**
3.1 Ferramentas do Pipeline 4
3.2 Ambientes e Infraestrutura 5
3.3 Estratégia de Branching e Versionamento 6
4. Identificação da Configuração **6**
4.1 Itens de Configuração 6
4.2 Convenção de Nomenclatura 6
4.3 Estrutura do Repositório 7
5. Integração Contínua (CI) **7**
5.1 Etapas do Pipeline de CI 7
5.2 Política de Pull Request e Revisão de Código 7
5.3 Critérios de Qualidade (Quality Gate) 7
6. Entrega e Implantação Contínua (CD) **7**
6.1 Etapas do Pipeline de CD 7
6.2 Estratégias de Implantação 8
6.3 Rollback e Recuperação 8
7. Infraestrutura como Código (IaC) **8**
7.1 Versionamento de Infraestrutura 8
7.2 Ambientes Efêmeros 8
8. Controle de Configuração e Mudança **9**
8.1 Processo de Solicitação de Mudança 9
8.2 Comitê de Controle de Mudança (CCM) 9
9. Baselines e Releases **9**
9.1 Geração Automática de Baseline 9
9.2 Notas de Versão (Release Notes) 9
10. Estimativa do Status de Configuração **9**
10.1 Monitoramento e Observabilidade 9
10.2 Métricas DORA 9
10.3 Relatórios e Auditorias 10
11. Marcos **10**
12. Treinamento e Recursos **10**
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros **10**
14. Quadro-Resumo do Pipeline CI/CD **10**

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do aplicativo GameTracker (Flutter), com ênfase em práticas DevOps, em particular Integração Contínua (CI) e Entrega Contínua (CD) com GitHub Actions. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação. Itens do modelo-base que não se aplicam ao projeto (ex.: Infraestrutura como Código, containers, servidores) permanecem descritos neste documento e estão marcados com "Nota: não está no projeto.".

1.1 Finalidade

Estabelecer um padrão a ser seguido pelos dois integrantes da equipe, garantindo rastreabilidade, versionamento e controle de mudanças do aplicativo GameTracker, apoiado por pipelines automatizados de build, teste e geração de pacote (APK) no GitHub Actions.

1.2 Escopo

Este PGC é destinado aos integrantes da equipe do GameTracker: Enzo Camelo (Desenvolvedor Líder) e camelo-dev (par de revisão), responsáveis pelo desenvolvimento e manutenção do aplicativo mobile GameTracker (Flutter, com pastas de plataforma android, ios, web, linux, macos e windows). Não há API/backend nem site próprio: os dados atuais são mock (lib/dados/jogos_mock.dart). Nota: não está no projeto. (aplicação web hospedada, API e publicação nas lojas de aplicativos).

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code). Nota: não está no projeto.
CCM	Comitê de Controle de Mudanças (aqui: Desenvolvedor Líder + par de revisão)
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate). Nota: não está no projeto.

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM).

Repositório de referência: https://github.com/lzcamargo/ongconecta (ONGConecta) e o PGC DevOps da ONGConecta (PDF), usados como base deste plano. O plano também está publicado em docs/plano-gcs.md.

1.5 Visão Geral

Neste plano a GCS é tratada como parte constante do pipeline de entrega: cada Pull Request revisado e testado gera evidências de rastreabilidade (logs de pipeline do GitHub Actions, histórico de PRs e APK gerado como artefato), reduzindo o esforço manual. Tags de versão geradas automaticamente e implantação em produção: Nota: não está no projeto.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto DevOps é responsabilidade compartilhada entre desenvolvimento e operações, apoiada por automação. Os papéis abaixo descrevem responsabilidades típicas; no GameTracker, os dois integrantes (Enzo Camelo e camelo-dev) acumulam todos os papéis.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Exercido por Enzo Camelo. Prioriza o backlog (GitHub Issues), autoriza mudanças de maior impacto e acompanha a entrega. Indicadores DORA: Nota: não está no projeto.
Engenheiro DevOps / Release Engineer	Exercido por Enzo Camelo. Projeta e mantém os pipelines de CI/CD (GitHub Actions), define a estratégia de branching e o versionamento. Registro de artefatos/imagens e IaC: Nota: não está no projeto.
Desenvolvedor(a)	Exercido por Enzo Camelo e camelo-dev. Integram código por Pull Requests, escrevem testes automatizados (flutter test), revisam os PRs um do outro e são corresponsáveis pela qualidade do que é integrado.
SRE / Operações *Site Reliability Engineering*	Nota: não está no projeto. O app não tem ambiente de produção com servidores a monitorar; SLIs/SLOs, postmortems e rollback operacional não são praticados.
QA / Automação de Testes	Exercido por Enzo Camelo e camelo-dev. Mantêm a suíte de testes automatizados (pasta test/) executada no CI. Testes de integração e de aceitação: Nota: não está no projeto.
Segurança (DevSecOps)	Exercido por Enzo Camelo. Acompanha os alertas e PRs do Dependabot e mantém segredos fora do repositório (.gitignore e .env.example). Análise estática de segurança (SAST): Nota: não está no projeto.
Comitê de Controle de Mudanças (CCM)	Composto por Enzo Camelo e camelo-dev. Atua apenas em mudanças de alto risco (ex.: dados pessoais, assinatura/applicationId do app, hotfix fora do fluxo padrão), registradas na Issue "Mudança de alto risco (CCM)"; mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas pelo projeto. O que consta no modelo-base e não é usado está marcado com "Nota: não está no projeto.":

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub (repositório público	Versionamento de código-fonte, pipelines como código e documentação

Categoria	Ferramenta(s) adotada(s)	Finalidade
	Enzo-Camelo/GameTracker)	
Orquestração de CI/CD	GitHub Actions (workflows ci.yml e release.yml)	Automação de build, testes, análise e implantação
Empacotamento	Flutter (flutter build apk). Docker/Buildpacks: Nota: não está no projeto.	Geração do APK (debug no CI; release a cada merge na main) publicado como artefato do GitHub Actions
Registro de artefatos	Nota: não está no projeto.	Não há registro de imagens/pacotes; o APK fica como artefato do GitHub Actions. Nota: não está no projeto.
Infraestrutura como Código	Nota: não está no projeto.	Não há infraestrutura a provisionar. Nota: não está no projeto.
Orquestração de containers	Nota: não está no projeto.	Não há serviços implantados a escalonar. Nota: não está no projeto.
Gestão de segredos	GitHub Secrets / Environments (HashiCorp Vault e AWS Secrets Manager: Nota: não está no projeto.)	Armazenamento de credenciais usadas nos pipelines. Nenhum segredo é usado hoje.
Monitoramento e Observabilidade	Nota: não está no projeto.	Não há monitoramento de ambientes. Nota: não está no projeto.
Gestão de dependências / SCA - Software Composition Analysis	Dependabot (ecossistema pub, atualização semanal) + pubspec.lock	Identificação de vulnerabilidades em bibliotecas de terceiros
Rastreamento de tarefas	GitHub Issues (templates de funcionalidade, defeito e mudança de alto risco)	Vinculação de commits/PRs a itens de trabalho

3.2 Ambientes e Infraestrutura

Os ambientes são representados pelos Environments do GitHub (staging e production). Provisionamento via Infraestrutura como Código: Nota: não está no projeto. Os Environments ainda não implantam em servidores.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Máquina local de cada desenvolvedor (flutter run) e execução do CI a cada Pull Request.	Abertura/atualização de Pull Request
Homologação / Staging	Environment staging do GitHub: ponto de validação após o merge. Deploy real e testes de aceitação em staging: Nota: não está no projeto.	Merge na branch main com CI verde
Produção	Environment production do GitHub, com Enzo Camelo como revisor obrigatório. Publicação para usuários finais/lojas: Nota: não está no projeto.	Aprovação de Enzo Camelo no Environment production após validação em staging

Execução: dispositivos e emuladores via Flutter. DNS, IPs, load balancer, banco de dados e cofre de segredos: Nota: não está no projeto.

3.3 Estratégia de Branching e Versionamento

Estratégia adotada: GitHub Flow / Trunk-Based Development, com branches de vida curta e a main sempre protegida:

- main - protegida: PR obrigatório, 1 aprovação do par de revisão, aprovações descartadas a cada novo commit, histórico linear, sem bypass e sem force-push; somente squash merge.
- feature/<issue>-<descrição-curta> - branches de curta duração para cada mudança (também fix/, chore/ e docs/).
- release/x.y (opcional) - Nota: não está no projeto.
- hotfix/<descrição> - Nota: não está no projeto. Correções seguem o fluxo normal de PR.

Versionamento: Versionamento Semântico (campo version do pubspec.yaml) e Conventional Commits (feat, fix, chore, docs, refactor…); o título do PR vira o commit na main (squash). Geração automática de changelog e da próxima versão pelo pipeline: Nota: não está no projeto.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, os seguintes artefatos também são itens de configuração formais, versionados no repositório Enzo-Camelo/GameTracker:

- Código-fonte do aplicativo (lib/, assets/, pastas de plataforma), testes automatizados (test/) e dependências (pubspec.yaml e pubspec.lock)
- Definições de pipeline como código (.github/workflows/ci.yml e release.yml), dependabot.yml, CODEOWNERS e templates de PR e de Issue
- Dockerfiles e manifests de containers - Nota: não está no projeto.
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes) - Nota: não está no projeto.
- Arquivos de configuração por ambiente (variáveis): o .env.example documenta os nomes; o .env real é ignorado pelo Git e segredos ficam fora do repositório. Feature flags - Nota: não está no projeto.
- Scripts de migração de banco de dados - Nota: não está no projeto. (N/A)
- Documentação técnica (README, plano de GCS em docs/plano-gcs.md, roteiro e evidências em docs/). ADRs - Nota: não está no projeto.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<issue>-<descrição-curta>	feature/12-filtro-por-plataforma
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(lista): adiciona filtro por plataforma
Tag de release	Versionamento Semântico vX.Y.Z	v1.1.0 (tags automáticas: Nota: não está no projeto.)
Imagem de container	Nota: não está no projeto.	Nota: não está no projeto. (o artefato gerado é o APK game-tracker-release-apk)
...	...	...

4.3 Estrutura do Repositório

Estrutura: /lib (telas, widgets, modelos, dados), /test, /assets, /android /ios /web /linux /macos /windows (plataformas Flutter), /.github (workflows, CODEOWNERS, templates, dependabot.yml), /docs (plano de GCS, roteiro, evidências), pubspec.yaml e pubspec.lock. /infra (IaC) e pasta /pipelines separada: Nota: não está no projeto.

5. Integração Contínua (CI)

Todo Pull Request para a branch main dispara automaticamente o workflow CI (ci.yml), que também roda em push na main. Nenhuma mudança deve ser integrada sem que o pipeline seja concluído com sucesso. Obrigatoriedade técnica do check build-and-test na proteção da main: ainda não configurada.

5.1 Etapas do Pipeline de CI

1. Checkout do código, instalação do Flutter (subosito/flutter-action, com cache) e restauração de dependências (flutter pub get).
2. Build de verificação do aplicativo (flutter build apk --debug).
3. Análise estática de código (flutter analyze) conforme o analysis_options.yaml.
4. Testes unitários e de widget (flutter test). Medição de cobertura mínima: Nota: não está no projeto.
5. Testes de integração. Nota: não está no projeto.
6. Análise de dependências (SCA) pelo Dependabot. Análise de segurança do código (SAST): Nota: não está no projeto.
7. Publicação do APK de release como artefato do GitHub Actions (release.yml, na main). Registro de imagens/artefatos versionado: Nota: não está no projeto.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação de outra pessoa (camelo-dev revisa os PRs de Enzo Camelo e vice-versa) antes do merge ("quatro olhos"). O GitHub não permite aprovar o próprio PR.
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente com pipeline de CI verde (quality gate). Hoje é combinação da equipe; o bloqueio técnico pelo check obrigatório ainda não está configurado.

5.3 Critérios de Qualidade (Quality Gate)

Limites adotados: flutter analyze sem erros, flutter test sem falhas e build do APK concluído. Meta de cobertura de testes e SAST: Nota: não está no projeto.

6. Entrega e Implantação Contínua (CD)

Após o CI, o workflow release.yml gera o APK de release a cada merge na main. A promoção entre ambientes (staging → production) com aprovação ainda não está implementada: os Environments existem, mas o pipeline de CD completo. Nota: não está no projeto.

6.1 Etapas do Pipeline de CD

8. Deploy automático em ambiente de Homologação/Staging. Nota: não está no projeto. (o Environment staging existe, sem deploy).
9. Execução de testes automatizados de aceitação/E2E em staging. Nota: não está no projeto.
10. Gate de aprovação para produção: aprovação manual de Enzo Camelo no Environment production. Nota: não está no projeto. (ainda sem workflow que o utilize).
11. Deploy em produção com estratégia definida (ver 6.2). Nota: não está no projeto.
12. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks). Nota: não está no projeto.
13. Geração automática de baseline/tag de release e notas de versão (changelog). Nota: não está no projeto. Hoje apenas o APK de release é gerado como artefato.

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade. Nota: não está no projeto.
Blue-Green	Duas versões (ambientes) coexistem; o tráfego é comutado de uma vez para a nova versão, permitindo rollback imediato. Nota: não está no projeto.
Canary Release	A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança. Nota: não está no projeto.
Feature Flags	Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy, reduzindo o risco de releases. Nota: não está no projeto.

6.3 Rollback e Recuperação

O rollback do aplicativo é feito pelo Git: reverter o PR (novo PR de revert) e gerar novo APK pelo release.yml. Rollback automatizado por pipeline, imagens em registro e IaC versionada: Nota: não está no projeto.

7. Infraestrutura como Código (IaC)

Nota: não está no projeto. O GameTracker não possui infraestrutura provisionada; esta seção consta por herança do modelo-base. Se passar a ter, deverá ser descrita como código versionado e submetida ao mesmo fluxo de PR e pipeline.

7.1 Versionamento de Infraestrutura

- Nota: não está no projeto. (Alterações de infraestrutura, se existirem, seguirão PR + revisão + pipeline, com "plan" automático em PR e "apply" após aprovação.)
- Nota: não está no projeto. (O estado da infraestrutura, se existir, deverá ser armazenado de forma centralizada e versionada.)

7.2 Ambientes Efêmeros

Nota: não está no projeto. Não há ambientes de revisão por Pull Request; a revisão é feita pelo código e pelo resultado do CI.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina seguem o fluxo padrão: Issue no GitHub → branch de feature → Pull Request → revisão do par → pipeline de CI → squash merge → release.yml (APK). Esse fluxo, por si, já constitui o registro formal e rastreável da mudança. O pipeline de CD completo (staging/produção): Nota: não está no projeto.

8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto risco, como tratamento de dados pessoais (LGPD), alteração de assinatura/applicationId do app e exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal. Alterações de infraestrutura e de esquema de banco de dados: Nota: não está no projeto.

Composição do CCM: Enzo Camelo (Desenvolvedor Líder) e camelo-dev (par de revisão). Acionamento: abertura de Issue com o template "Mudança de alto risco (CCM)", contendo plano de rollback; a decisão é registrada nos comentários.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Neste plano, cada implantação bem-sucedida em produção deve gerar uma baseline: tag de versão (SemVer) associada ao commit e ao APK publicado. Nota: não está no projeto. Hoje apenas o APK de release é gerado como artefato a cada merge na main; imagem em registro e estado de infraestrutura também não fazem parte do projeto.

9.2 Notas de Versão (Release Notes)

As notas de release devem ser geradas a partir dos Conventional Commits desde a última tag. Nota: não está no projeto. O histórico da main (um commit por PR, via squash) serve de registro.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Monitoramento contínuo dos ambientes (métricas, logs e alertas). Nota: não está no projeto. O app não tem ambiente de produção com servidor a monitorar.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção. Nota: não está no projeto.
Lead Time for Changes	Tempo entre o commit e a implantação em produção. Nota: não está no projeto.

Métrica	Descrição
Change Failure Rate	Percentual de implantações que resultam em falha em produção. Nota: não está no projeto.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha. Nota: não está no projeto.

10.3 Relatórios e Auditorias

O histórico do GitHub Actions (builds e testes), os PRs aprovados e os logs do Git constituem o principal insumo de auditoria de GCS, dispensando relatórios manuais.

11. Marcos

Concluídos: configuração do repositório e da proteção da main; plano de GCS publicado; CI (build-and-test) e geração do APK de release; Environments staging e production criados; Dependabot ativo. Previstos: check do CI obrigatório na main; CD com staging, aprovação em production, baseline/tag e rollback; governança de dependências e rastreabilidade. Implantação em servidores, publicação em lojas e IaC: Nota: não está no projeto.

12. Treinamento e Recursos

Ferramentas: Git, GitHub (Actions, Issues, Environments, Dependabot), Flutter SDK e VS Code. Pessoal: Enzo Camelo e camelo-dev. Treinamento: README, docs/roteiro-github.md e o repositório de referência ONGConecta. Ferramentas externas de monitoramento e publicação em lojas: Nota: não está no projeto.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas de terceiros (cupertino_icons, fl_chart, flutter_lints) também são itens de configuração, versionadas explicitamente em pubspec.yaml e pubspec.lock (lockfile).

- Análise de vulnerabilidades em dependências pelos alertas do Dependabot. Etapa de SCA dedicada no pipeline de CI: Nota: não está no projeto.
- Atualizações de dependências propostas automaticamente pelo Dependabot (semanal, ecossistema pub) e submetidas ao mesmo fluxo de PR.

O projeto não prevê subcontratados nem fornecedores externos de software. Serviços de terceiros em produção (e-mail, pagamentos): Nota: não está no projeto.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Análise estática (flutter analyze), testes (flutter test) e build do APK de verificação. Análise de segurança (SAST) e publicação de imagem: Nota: não está no projeto.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação do par de revisão + CI verde
CD - Staging	Deploy automático em homologação + testes de aceitação. Nota: não está no projeto. (Environment staging criado, sem deploy.)	Merge na branch principal
Gate de Release	Aprovação de Enzo Camelo (revisor obrigatório do Environment production). Nota: não está no projeto. (ainda sem deploy.)	Sucesso dos testes em staging
CD - Produção	Deploy, smoke tests e geração de baseline/tag. Nota: não está no projeto. Hoje o release.yml apenas gera o APK de release como artefato.	Aprovação do gate de release

Nota: este plano foi adaptado ao contexto real do projeto GameTracker (equipe de 2 pessoas, aplicativo Flutter, GitHub Actions), mantendo os quatro pilares da GCS: identificação, controle de versão, controle de mudanças e auditoria/status, suportados por automação. Tudo o que está marcado com "Nota: não está no projeto." consta neste documento por herança do modelo-base e do PGC da ONGConecta, mas não faz parte do repositório.